

Exercício 15

```

a = [10, 20, 30]
b = a

b.append(40)

print("a:", a)
print("b:", b)
print("id(a):", id(a))
print("id(b):", id(b))

a: [10, 20, 30, 40]
b: [10, 20, 30, 40]
id(a): 132441683881472
id(b): 132441683881472

```

Questão 1

a: [10, 20, 30, 40]

b: [10, 20, 30, 40]

id(a): 132441683922176

id(b): 132441683922176

Questão 2 Sim — id(a) == id(b) vai dar True

Questão 3 b = a não cria uma lista nova, só faz b apontar pra mesma lista que a já apontava. Por isso o b.append(40) também "aparece" em a: não existem duas listas, existe uma lista só com dois nomes (rótulos) apontando pra ela.

Questão 4 O b seria uma lista independente, o .append do "b" não afetaria o "a"

Exercício 16

```

class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

n1 = Node("A")
n2 = Node("B")
n3 = Node("C")

# Conectando: A → B → C → None
n1.next = n2
n2.next = n3
n3.next = None # já é o padrão, mas deixa explícito

# Percorrendo a estrutura
atual = n1
while atual is not None:
    print(atual.value)
    atual = atual.next

```

1. Dentro do while, o código imprime atual.valor, mas nunca atualiza o atual para o próximo nó. Ele fica sempre apontando para n1.
2. A condição do loop é atual is not None. Como atual nunca muda (continua sendo sempre n1), essa condição nunca se torna falsa — é um loop infinito, imprimindo 10 pra sempre.
3. Dentro do while, depois do print, precisa avançar o ponteiro:

python atual = atual.proximo

4.10

20

30

Exercício Desafio!

```

class Node:
    __slots__ = ('nome', 'prioridade', 'proximo') # economiza memória por nó

    def __init__(self, nome, prioridade=False):

```

```
self.nome = nome
self.prioridade = prioridade
self.proximo = None

class FilaAtendimento:
    def __init__(self):
        self.cabeca = None
        self.cauda = None
        self.ultima_prioridade = None
        self.tamanho = 0

    def inserir(self, nome, prioridade=False):
        novo = Node(nome, prioridade)
        self.tamanho += 1

        if self.cabeca is None:
            self.cabeca = self.cauda = novo
            if prioridade:
                self.ultima_prioridade = novo
            return

        if prioridade:
            if self.ultima_prioridade is None:
                novo.proximo = self.cabeca
                self.cabeca = novo
            else:
                novo.proximo = self.ultima_prioridade.proximo
                self.ultima_prioridade.proximo = novo
                if self.ultima_prioridade is self.cauda:
                    self.cauda = novo
                self.ultima_prioridade = novo
        else:
            self.cauda.proximo = novo
            self.cauda = novo

    def atender(self):
        """Remove e retorna o nome do paciente da cabeça - O(1)."""
        if self.cabeca is None:
            return None
        removido = self.cabeca
        self.cabeca = removido.proximo
        if self.cabeca is None:
            self.cauda = None
        if self.ultima_prioridade is removido:
            self.ultima_prioridade = None
        self.tamanho -= 1
        return removido.nome

    def percorrer(self):
        atual = self.cabeca
        posicao = 1
        while atual is not None:
            tipo = "Prioridade" if atual.prioridade else "Normal"
            print(f"{posicao}. {atual.nome} ({tipo})")
            atual = atual.proximo
            posicao += 1

# Teste
fila = FilaAtendimento()
fila.inserir("Ana", prioridade=False)
fila.inserir("Bruno", prioridade=False)
fila.inserir("Carlos", prioridade=True)

fila.percorrer()
```

